

A - Phone Charging

Time limit: 2 seconds Memory limit: 2 GB

You want to charge your phone. Its current battery level is x percent.

If the current battery level is less than 80 percent, increasing it by 1 percent takes a seconds. If the current battery level is at least 80 percent, increasing it by 1 percent takes b seconds.

Find the total number of seconds needed for the battery level to reach 100 percent.

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

Each test case consists of a single line containing three integers x , a , and b — the current battery level and the times needed to increase it by 1 percent below and at or above 80 percent, respectively.

Output Format

For each test case, print a single integer — the number of seconds needed for the battery level to reach 100 percent.

Constraints

- $1 \leq t \leq 100$
- $0 \leq x \leq 100$
- $1 \leq a < b \leq 100$

Sample Tests

Input	Output
5	120
0 1 2	207
79 7 10	160
80 3 8	100
99 1 100	0
100 99 100	

Explanation

In the first test case, charging from 0 percent to 80 percent takes 1 second each, for 80 seconds in total. Charging from 80 to 100 takes 2 seconds per percent, for $2 \cdot 20 = 40$ seconds in total.

The sum is $80 + 40 = 120$ seconds.

In the second test case, charging from 79 percent to 80 percent takes 7 seconds. Each of the remaining 20 percent takes 10 seconds each, so the total time is $7 + 20 \cdot 10 = 207$ seconds.

In the fifth test case, the phone is already at 100 percent, so the required time is 0 seconds.

B - Cyclic Shift

Time limit: 2 seconds Memory limit: 2 GB

You are given an array $a = [a_1, a_2, \dots, a_n]$ of n integers. For any array $b = [b_1, b_2, \dots, b_n]$ of length n , define

$$f(b) = \max_{1 \leq i < n} |b_i - b_{i+1}|.$$

A **cyclic shift** of a is an array of the form

$$[a_k, a_{k+1}, \dots, a_n, a_1, a_2, \dots, a_{k-1}]$$

for some $1 \leq k \leq n$. In particular, choosing $k = 1$ leaves the array unchanged.

Find the minimum possible value of $f(b)$ over all cyclic shifts b of a .

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

The first line of each test case contains a single integer n — the length of the array.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ — the elements of the array.

Output Format

For each test case, print a single integer — the minimum possible value of $f(b)$ over all cyclic shifts b of a .

Constraints

- $1 \leq t \leq 10^5$
- $2 \leq n \leq 2 \cdot 10^5$
- $1 \leq a_i \leq 10^6$
- It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Sample Tests

Input	Output
4	9
2	2
1 10	0
4	99
1 4 2 3	
3	
7 7 7	
6	
1 100 1 100 1 100	

Explanation

Test Case 1:

The given array is $[1, 10]$. Its two cyclic shifts are the arrays:

1. $b = [1, 10]$
 - $f([1, 10]) = \max_{1 \leq i < 2} |b_i - b_{i+1}| = \max\{|b_1 - b_2|\} = \max\{|1 - 10|\} = 9$
2. $b = [10, 1]$
 - $f([10, 1]) = \max_{1 \leq i < 2} |b_i - b_{i+1}| = \max\{|b_1 - b_2|\} = \max\{|10 - 1|\} = 9$

Both cyclic shifts have their f -value 9. So the minimum f -value over all the cyclic shifts is 9. So the answer is 9.

Test Case 2:

The given array is $[1, 4, 2, 3]$. Its four cyclic shifts are the arrays:

1. $b = [1, 4, 2, 3]$
 - $f([1, 4, 2, 3]) = \max_{1 \leq i < 4} |b_i - b_{i+1}| = \max\{|1 - 4|, |4 - 2|, |2 - 3|\} = \max\{3, 2, 1\} = 3$
2. $b = [4, 2, 3, 1]$
 - $f([4, 2, 3, 1]) = \max_{1 \leq i < 4} |b_i - b_{i+1}| = \max\{|4 - 2|, |2 - 3|, |3 - 1|\} = \max\{2, 1, 2\} = 2$
3. $b = [2, 3, 1, 4]$
 - $f([2, 3, 1, 4]) = \max_{1 \leq i < 4} |b_i - b_{i+1}| = \max\{|2 - 3|, |3 - 1|, |1 - 4|\} = \max\{1, 2, 3\} = 3$
4. $b = [3, 1, 4, 2]$
 - $f([3, 1, 4, 2]) = \max_{1 \leq i < 4} |b_i - b_{i+1}| = \max\{|3 - 1|, |1 - 4|, |4 - 2|\} = \max\{2, 3, 2\} = 3$

The f -values of the cyclic shifts are 3, 2, 3, 3. So the minimum f -value over all the cyclic shifts is 2, attained by $[4, 2, 3, 1]$. So the answer is 2.

Test Case 3:

The given array is $[7, 7, 7]$. Its three cyclic shifts are the arrays:

1. $b = [7, 7, 7]$
 - $f([7, 7, 7]) = \max_{1 \leq i < 3} |b_i - b_{i+1}| = \max\{|7 - 7|, |7 - 7|\} = \max\{0, 0\} = 0$
2. $b = [7, 7, 7]$
 - $f([7, 7, 7]) = \max_{1 \leq i < 3} |b_i - b_{i+1}| = \max\{|7 - 7|, |7 - 7|\} = \max\{0, 0\} = 0$
3. $b = [7, 7, 7]$
 - $f([7, 7, 7]) = \max_{1 \leq i < 3} |b_i - b_{i+1}| = \max\{|7 - 7|, |7 - 7|\} = \max\{0, 0\} = 0$

Since all elements are equal, every cyclic shift is the same array, and every adjacent difference is 0. So the minimum f -value over all the cyclic shifts is 0. So the answer is 0.

C - Furious Farming

Time limit: 2 seconds Memory limit: 2 GB

After a long think about what career you would like to pursue, you have decided to take up farming. Unfortunately, you overlooked a crucial detail - a farmer's livelihood depends heavily on rain, and the rain this year has been quite poor! So, you decide to take things into your own hands.

You have n fields, numbered 1 to n from left to right.

Using the power of cloud seeding, you have managed to create m rainclouds. Initially, the i -th raincloud is directly above field number x_i .

It is guaranteed that all x_i are pairwise distinct.

You have also managed to, somehow, figure out a way to move the clouds by creating a strong enough wind system. However, the system is primitive, and so on each day, you can only either move *all* clouds left by one step, or move *all* clouds right by one step.

Note that all clouds move simultaneously in the same direction on each day.

For your farming venture to be a success, every field must receive some rain - that is, for each $1 \leq i \leq n$, there must exist some day such that there is a raincloud above field i .

Given that you choose the directions for cloud movements optimally, find the minimum number of days needed to ensure that every field is visited by a raincloud at least once.

Note that this includes the initial configuration as well, i.e. every field x_i is considered visited even before any days have passed.

Also note that it is allowed for clouds to leave the interval $[1, n]$ if necessary.

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

The first line of each test case contains two integers n and m — the number of fields and the number of clouds.

The second line of each test case contains m distinct integers $x_1, x_2, \dots, x_m$ — the initial positions of the clouds.

Output Format

For each test case, print a single integer — the minimum number of days needed to ensure that every field is visited by a raincloud at least once.

Constraints

- $1 \leq t \leq 2 \cdot 10^4$
- $1 \leq m \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq n$
- The x_i are all distinct within each test case.
- It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Sample Tests

Input	Output
5	3
5 2	0
4 2	12
5 5	8
3 1 5 2 4	7
10 1	
4	
10 2	
10 1	
12 3	
10 3 4	

Explanation

In the first test case, the clouds are at positions $[2, 4]$ initially. Consider the following sequence of movements:

- Day 1: Move the clouds one step to the right. Now they're at $[3, 5]$.
- Day 2: Move the clouds one step to the left. Now they're at $[2, 4]$.
- Day 3: Move the clouds one step to the left again. Now they're at $[1, 3]$.

There are 5 fields, and by the end of the third day all of them have been visited by a raincloud at least once. You can verify that this is the best you can do, and hence the answer is 3.

In the second test case, every field already has a raincloud over it, so nothing needs to be done and the answer is 0.

D - Curio Packing

Time limit: 2 seconds Memory limit: 2 GB

There are n curios arranged from left to right. Each curio is made of either glass or iron. A glass curio has weight 1, and an iron curio has weight 2.

You would like to buy all n curios, in order from left to right. You have two bags with you to hold the curios.

Immediately after buying a curio, you place it in either bag, on top of all the curios already in that bag.

A glass curio breaks if, at any moment, the total weight of the curios above it in the same bag becomes strictly greater than k .

Iron curios never break.

Determine whether it is possible to buy all the curios and place them in the bags without breaking any glass curio.

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

The first line of each test case contains two integers n and k — the number of curios and the maximum weight that can be placed above a glass curio without breaking it.

The second line of each test case contains a string s of length n , consisting only of the characters **G** and **I**. If $s_i = \text{G}$, the i -th curio is made of glass; if $s_i = \text{I}$, it is made of iron.

Output Format

For each test case, print **YES** if it is possible to buy all the curios without breaking any glass curio, and **NO** otherwise.

You can print each letter in any case (uppercase or lowercase). For example, **YES**, **Yes**, **yes**, and **yEs** are all accepted.

Constraints

- $1 \leq t \leq 4 \cdot 10^4$
- $1 \leq n \leq 2000$
- $1 \leq k \leq 2n$
- It is guaranteed that the sum of n^2 over all test cases does not exceed 2000^2 .

Sample Tests

Input	Output
6	YES
1 1	YES
G	NO
3 1	YES
GII	NO
4 1	YES
GGGI	
4 1	
GGIG	
6 2	
GGGGGI	
6 3	
GGGGGI	

Explanation

In the first test case, there is only one glass curio. No weight is placed above it, so the answer is **YES**.

In the second test case, you can place the glass curio in one bag and both iron curios in the other bag.

E - Side Hustle

Time limit: 2 seconds Memory limit: 2 GB

You are a poor graduate student trying to earn money by doing odd jobs.

One day, you find n people standing in a line, each holding one token. Person i holds token p_i , where p is a permutation of $1, 2, \dots, n$.

A permutation of $1, 2, \dots, n$ is an array of length n that contains every integer from 1 to n in some order.

You may choose two distinct people and swap the tokens they hold.

Each swap costs c coins. You can perform any number of swaps, possibly zero.

After all your swaps, if person i holds token i , they will happily give you a_i coins.

Your *profit* is the total amount you receive minus the total cost of your swaps. Find the maximum possible profit.

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

The first line of each test case contains two integers n and c — the number of people and the cost of one swap.

The second line contains n integers $p_1, p_2, \dots, p_n$ — the initial token held by each person. It is guaranteed that p is a permutation of $1, 2, \dots, n$.

The third line contains n integers $a_1, a_2, \dots, a_n$ — the reward from person i if they hold token i after all the swaps.

Output Format

For each test case, print a single integer — the maximum possible profit.

Constraints

- $1 \leq t \leq 10^5$
- $1 \leq n \leq 5 \cdot 10^5$
- $1 \leq c \leq 10^9$
- p is a permutation of $1, \dots, n$.
- $1 \leq a_i \leq 10^9$
- It is guaranteed that the sum of n over all test cases does not exceed $5 \cdot 10^5$.

Sample Tests

Input	Output
4	7
3 10	2
1 3 2	15
7 3 4	14
2 10	
2 1	
6 6	
3 5	
2 3 1	
20 1 1	
5 7	
2 3 1 5 4	
8 6 6 4 11	

Explanation

In the first test case, performing no swaps gets you the first person's reward, for a total profit of 7.

In the second test case, one swap fixes both people and gives a profit of $12 - 10 = 2$.

In the third test case, swapping once to make the first person correct gives a profit of $20 - 5 = 15$.

In the fourth test case, you can swap the tokens in this order - (Person 1, Person 3), (Person 2, Person 3), (Person 4, Person 5). After these swaps, all the 5 people have their own tokens. So you get a reward of $8 + 6 + 6 + 4 + 11 = 35$. And your cost was $3 \cdot 7 = 21$. So your total profit is $35 - 21 = 14$. You can verify that this is the best possible.

F - Take the L

Time limit: 2 seconds Memory limit: 2 GB

You are given n points on the 2D plane. Their x -coordinates are pairwise distinct, and their y -coordinates are pairwise distinct.

For each point (x_i, y_i) , you must choose one of the following two pairs of rays, both starting at that point:

- a horizontal ray extending to the right and a vertical ray extending upward; or
- a horizontal ray extending to the left and a vertical ray extending downward.

Determine whether it is possible to choose a pair of rays for every point so that no ray from one point intersects any ray from another point.

Note that the two rays from the same point meet at their common starting point - this intersection is allowed.

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

The first line of each test case contains a single integer n — the number of points.

Each of the next n lines contains two integers x_i and y_i — the coordinates of the i -th point.

Output Format

For each test case, print YES if it is possible to choose the rays as required, and NO otherwise.

You can print each letter in any case (uppercase or lowercase). For example, YES, Yes, yes, and yEs are all accepted.

Constraints

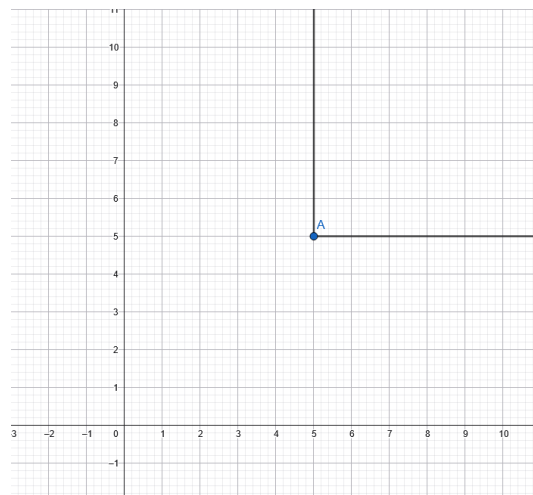
- $1 \leq t \leq 3 \cdot 10^4$
- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i, y_i \leq 10^9$
- It is guaranteed that the x -coordinates are pairwise distinct within each test case, and similarly the y -coordinates are pairwise distinct within each test case.
- It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Sample Tests

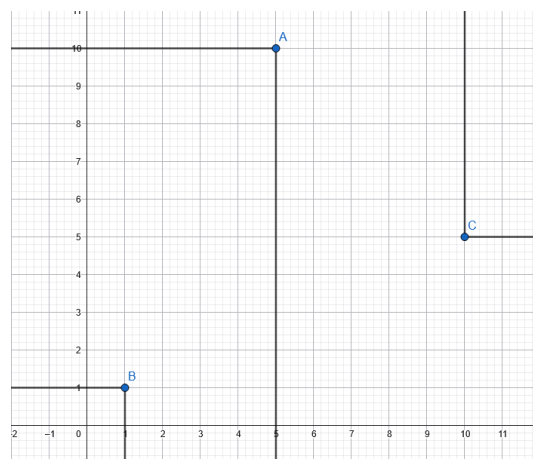
Input	Output
5	YES
1	YES
5 5	NO
3	NO
5 10	YES
1 1	
10 5	
3	
10 1	
1 10	
5 5	
4	
10 7	
1 4	
7 10	
4 1	
6	
6 3	
1 6	
10 5	
5 8	
3 1	
8 10	

Explanation

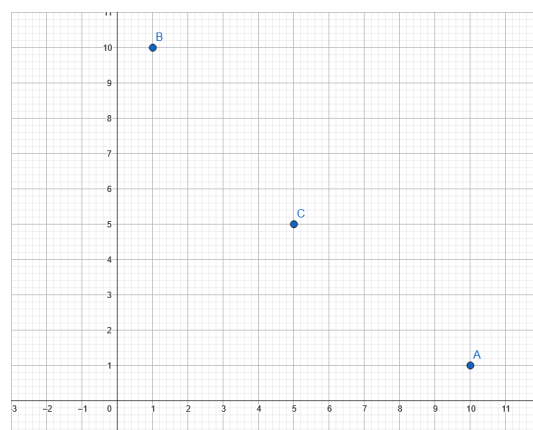
In the first test case, there is only one point, so either pair of rays is a valid choice. For example:



In the second test case, here is one valid solution:



In the third test case, these are the points. It can be verified that no valid solution exists.



G - Team Formation Tactics

Time limit: 2 seconds Memory limit: 2 GB

There are $3n$ students who want to participate in ICPC from your college. The rating of the i -th student is a_i . As their coach, your task is to divide all the students into n teams, each consisting of exactly 3 students.

How strong a team is often depends on the strongest member(s) of the team. So, you define the **strength** of a team of three students with ratings (x, y, z) , where $x \geq y \geq z$, to be

$$\max(6x, 4x + 4y, 3x + 3y + 3z)$$

Find the maximum possible sum of the strengths of all n teams.

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

The first line of each test case contains a single integer n — the number of teams.

The second line of each test case contains $3n$ integers $a_1, a_2, \dots, a_{3n}$ — the ratings of the students.

Output Format

For each test case, print a single integer — the maximum possible sum of the strengths of all teams.

Constraints

- $1 \leq t \leq 3 \cdot 10^4$
- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq a_i \leq 10^9$
- It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Sample Tests

Input	Output
5	60
1	64
10 1 1	81
1	34
10 6 1	482
1	
10 9 8	
2	
2 1 3 1 2 1	
3	
22 13 13 16 7 30 17 13 19	

Explanation

In the first test case of the example, there is only one team. Its strength is $\max(60, 44, 36) = 60$.

In the second test case, the strength of the only team is $\max(60, 64, 51) = 64$.

In the fourth test case, one optimal solution is as follows:

- Put students 1, 4, 5 with ratings $(2, 1, 2)$ in a single team.
The strength of this team is $\max(12, 16, 15) = 16$.
- That leaves students 2, 3, 6 to form the other team, with ratings $(1, 3, 1)$.
The strength of this team is $\max(18, 16, 15) = 18$.

The sum of strengths is $16 + 18 = 34$, and it can be verified that this is optimal.

H - Random Merging

Time limit: 2 seconds Memory limit: 2 GB

You are given an array a containing n integers.

Consider the following process.

Initially, $s = 0$. While the array contains more than one element, perform the following steps:

1. Choose an index i uniformly at random among all indices satisfying $1 \leq i < |a|$.
Here, $|a|$ denotes the current length of a .
2. Add $a_i + a_{i+1}$ to s .
3. Remove the elements a_i and a_{i+1} from the array, and insert their sum in the same location.
That is, the array becomes $[a_1, \dots, a_{i-1}, a_i + a_{i+1}, a_{i+2}, \dots]$.

The choice of index at each step is made independently of previous choices, given the current array.

Find the expected value of s modulo 998244353.

Formally, let p/q be the expected value written as an irreducible fraction. It can be shown that q is not divisible by 998244353. You must output $p \cdot q^{-1} \bmod 998244353$, where q^{-1} is the multiplicative inverse of q modulo 998244353.

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

The first line of each test case contains a single integer n — the length of the array.

The second line of each test case contains n space-separated integers $a_1, a_2, \dots, a_n$ — the elements of the array.

Output Format

For each test case, print a single integer — the expected value of s modulo 998244353.

Constraints

- $1 \leq t \leq 4 \cdot 10^4$
- $1 \leq n \leq 2 \cdot 10^5$
- $0 \leq a_i < 998244353$
- It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Sample Tests

Input	Output
5	0
1	12
42	10
2	499122178
5 7	166374086
3	
1 2 3	
3	
1 0 0	
4	
6 1 4 2	

Explanation

In the first test case, the array contains only one element, so no operation is performed and the answer is 0.

In the second test case, the only operation adds $5 + 7 = 12$ to s .

In the third test case, the two possible total values of s are 9 and 11, each with probability $1/2$. Therefore, the expected value is 10.

I - Mountain Medians

Time limit: 2 seconds Memory limit: 2 GB

A permutation of length n is an array of length n that contains each integer from 1 to n exactly once.

A permutation p of length n is called **mountain-shaped** if there exists an index k ($1 \leq k \leq n$) such that

$$p_1 < p_2 < \dots < p_k > p_{k+1} > \dots > p_n$$

In particular, a strictly increasing permutation and a strictly decreasing permutation are also considered to be mountain-shaped.

You are given an array a of length n .

Count the number of mountain-shaped permutations p of length n such that, for every index i ($1 \leq i \leq n$), **at least one** of the following conditions holds:

- $a_i = \text{med}(p_1, p_2, \dots, p_i)$;
- $a_i = \text{med}(p_i, p_{i+1}, \dots, p_n)$.

Here, med denotes the median.

Formally, for a non-empty multiset of m numbers, sort its elements as $b_1 \leq b_2 \leq \dots \leq b_m$. Its **median** is $b_{\lfloor (m+1)/2 \rfloor}$. In particular, for even m , this is the smaller of the two middle elements.

Since the answer can be large, print it modulo 998244353.

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

The first line of each test case contains a single integer n — the length of the array.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ — the elements of the array.

Output Format

For each test case, print a single integer — the number of valid permutations modulo 998244353.

Constraints

- $1 \leq t \leq 10^5$
- $1 \leq n \leq 5000$
- $1 \leq a_i \leq n$
- It is guaranteed that the sum of n^2 over all test cases does not exceed 5000^2 .

Sample Tests

Input	Output
5	1
1	2
1	0
2	8
1 1	12
3	
1 2 1	
4	
2 3 3 2	
5	
3 3 4 3 3	

Explanation

In the second test case of the example, both permutations $[1, 2]$ and $[2, 1]$ satisfy the conditions, so the answer is 2.

In the fourth test case, all $2^3 = 8$ mountain-shaped permutations of length 4 satisfy the conditions.

J - Off With Their Heads

Time limit: 2 seconds Memory limit: 2 GB

You are given a multiset of binary strings, each of length 2. It contains c_{00} copies of 00, c_{01} copies of 01, c_{10} copies of 10, and c_{11} copies of 11. Let $n = c_{00} + c_{01} + c_{10} + c_{11}$.

While the multiset contains more than one string, you must perform the following operation:

1. Choose two distinct occurrences of strings A and B in the multiset, in this order. Their contents may be identical.
2. Concatenate A and B , and delete the first character of the resulting string.
3. Remove the chosen occurrences of A and B from the multiset and insert the resulting string.

After exactly $n - 1$ operations, one string remains. Find the lexicographically smallest possible final string.

A binary string p is said to be lexicographically smaller than another binary string q if one of the two following conditions holds:

- p is a prefix of q that is not q itself; or
- There exists an index i such that $p_j = q_j$ for $1 \leq j < i$, and also $p_i = 0$ and $q_i = 1$.

Input Format

The first line contains a single integer t — the number of test cases. The description of the test cases follows.

Each test case consists of a single line containing four integers $c_{00}, c_{01}, c_{10}, c_{11}$ — the numbers of copies of 00, 01, 10, and 11, respectively.

Output Format

For each test case, print a single string — the lexicographically smallest possible final string.

Constraints

- $1 \leq t \leq 2 \cdot 10^4$
- $0 \leq c_{00}, c_{01}, c_{10}, c_{11} \leq 3 \cdot 10^5$
- In each test case, $c_{00} + c_{01} + c_{10} + c_{11} \geq 1$
- The sum of $c_{00} + c_{01} + c_{10} + c_{11}$ over all test cases does not exceed $3 \cdot 10^5$.

Sample Tests

Input	Output
4	00
1 0 0 0	011
0 0 1 1	00001
1 1 1 1	010101
0 3 0 2	

Explanation

In the first test case of the example, the multiset contains only 00, so no operation is performed.

In the second test case, choosing $A = 10$ and $B = 11$ produces 011, which is the lexicographically smallest possible final string.